

# myRTOS

## UML Diagram Reference

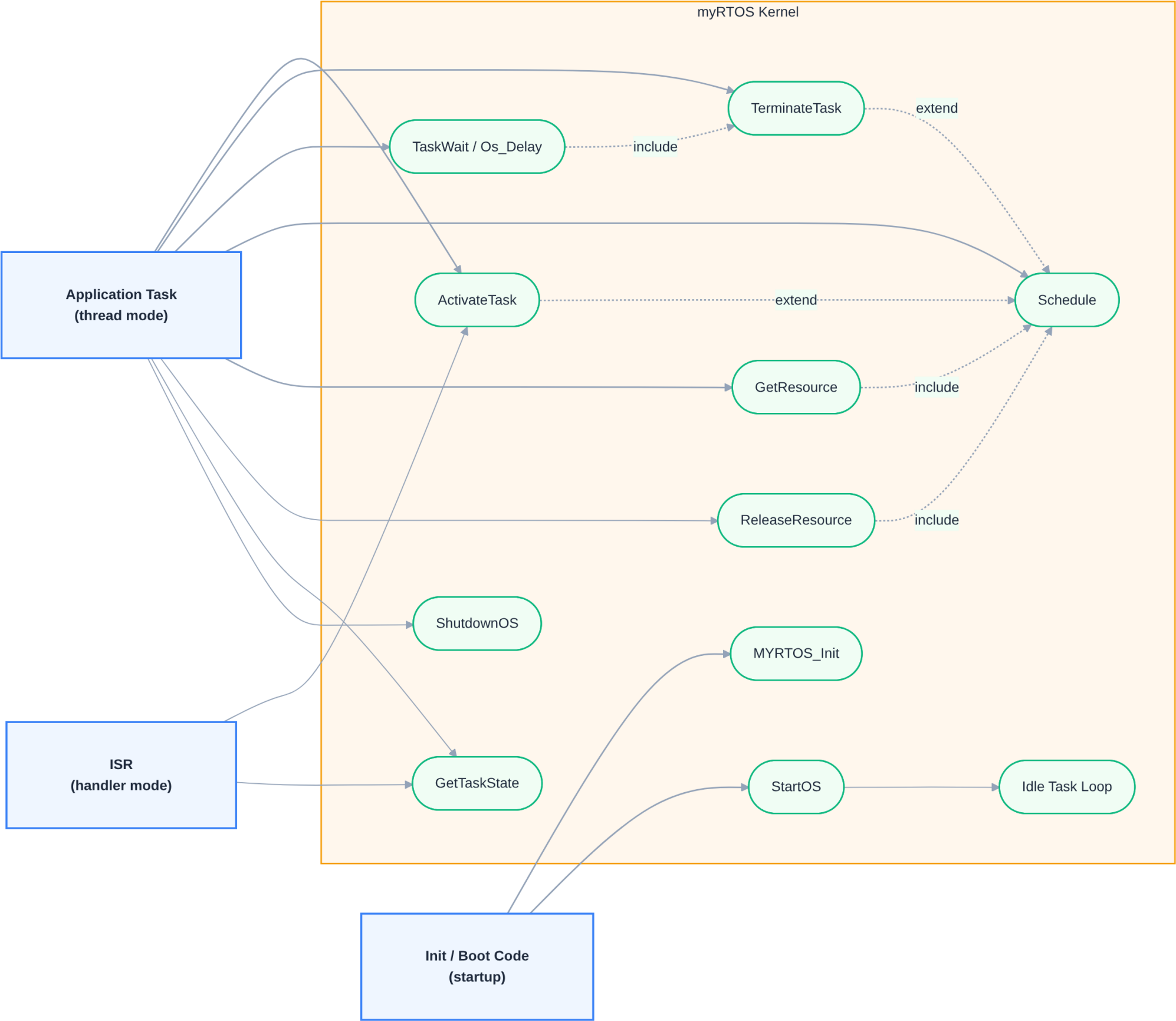
Cortex-M real-time kernel — Scheduler, OS API, FIFO ready queue, CortexMx porting layer

### Contents

|   |                                                                     |
|---|---------------------------------------------------------------------|
| 1 | Use Case Diagram                                                    |
| 2 | Class Diagram                                                       |
| 3 | Sequence Diagram — Start_OS (Boot)                                  |
| 4 | Sequence Diagram — ActivateTask (thread mode)                       |
| 5 | Sequence Diagram — TerminateTask + PendSV                           |
| 6 | Sequence Diagram — SysTick Tick (Preemption + Wait Expiry)          |
| 7 | Sequence Diagram — GetResource / ReleaseResource (Priority Ceiling) |
| 8 | Sequence Diagram — TaskWait / Os_Delay                              |

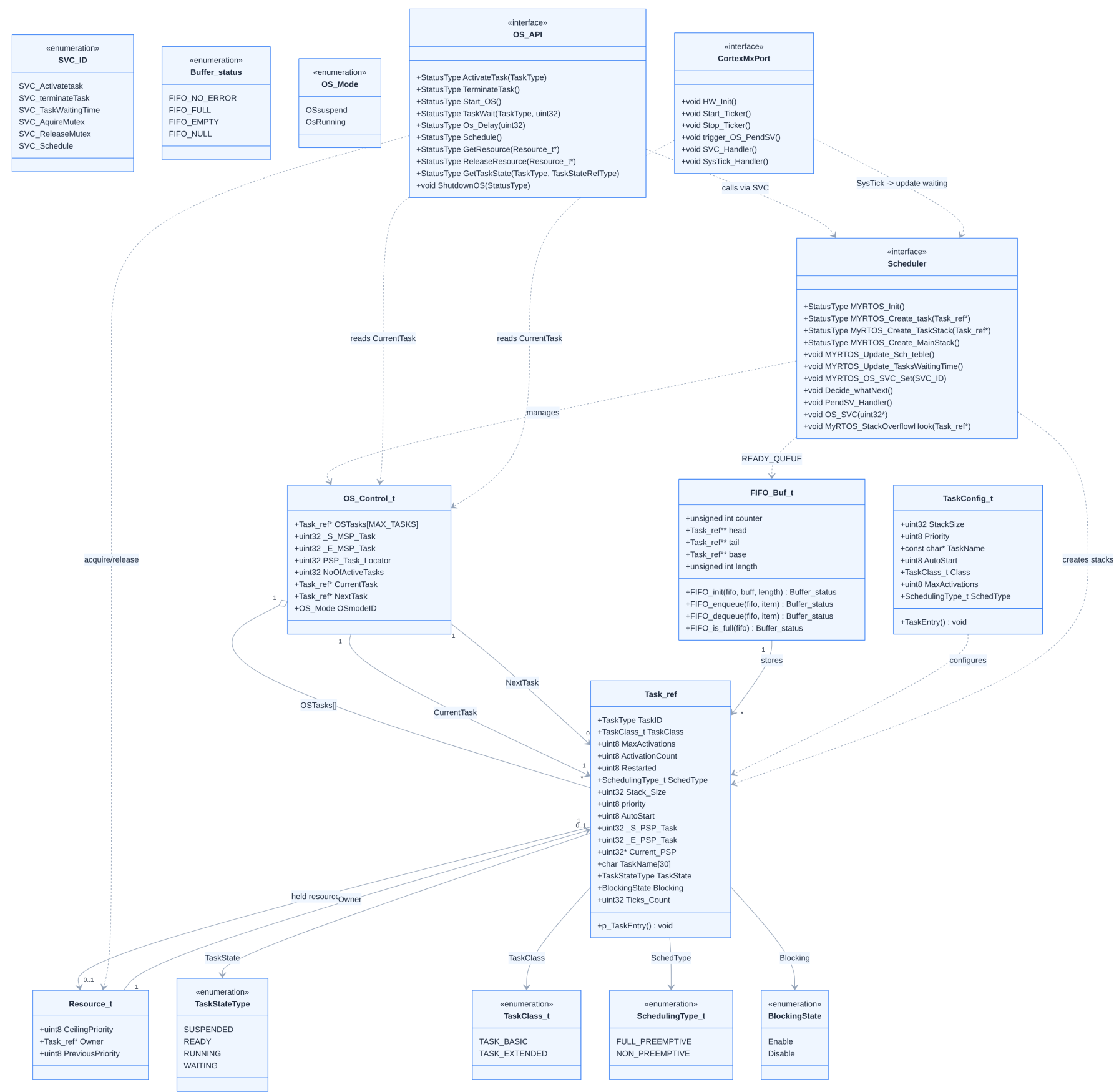
# Use Case Diagram

Actors (Application Task, ISR, Boot Code) and the 11 kernel services they invoke (StartOS, ActivateTask, TerminateTask, TaskWait / Os\_Delay, Schedule, GetResource, ReleaseResource, GetTaskState, ShutdownOS, MYRTOS\_Init, Idle Task) with include / extend relationships.



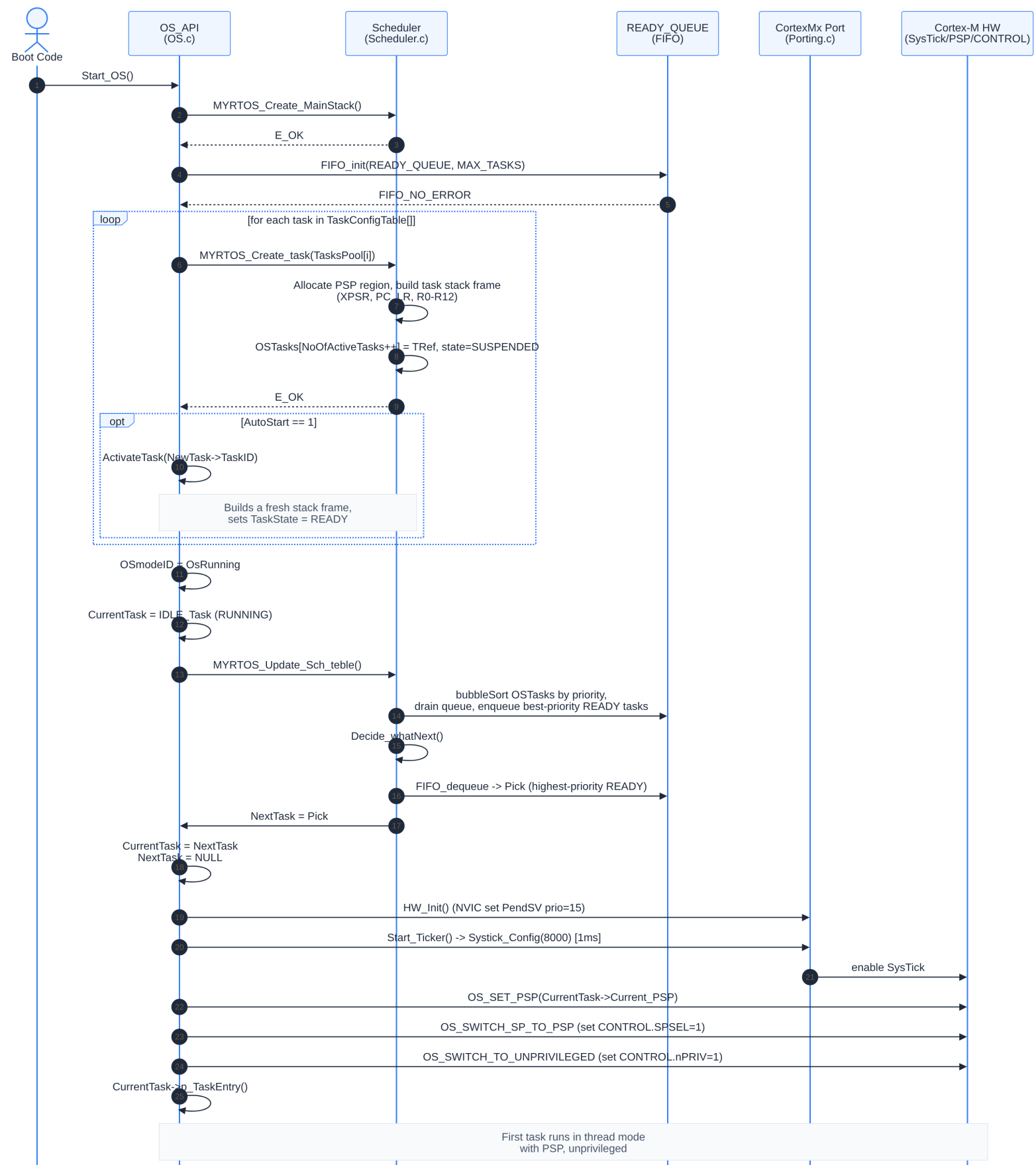
# Class Diagram

All enums (TaskStateType, SVC\_ID, TaskClass\_t, SchedulingType\_t, OS\_Mode, BlockingState), the TCB (Task\_ref), OS\_Control\_t kernel state, Resource\_t (priority-ceiling), TaskConfig\_t configuration, FIFO\_Buf\_t ready-queue, and the three module pseudo-classes Scheduler / OS\_API / CortexMxPort with their associations.



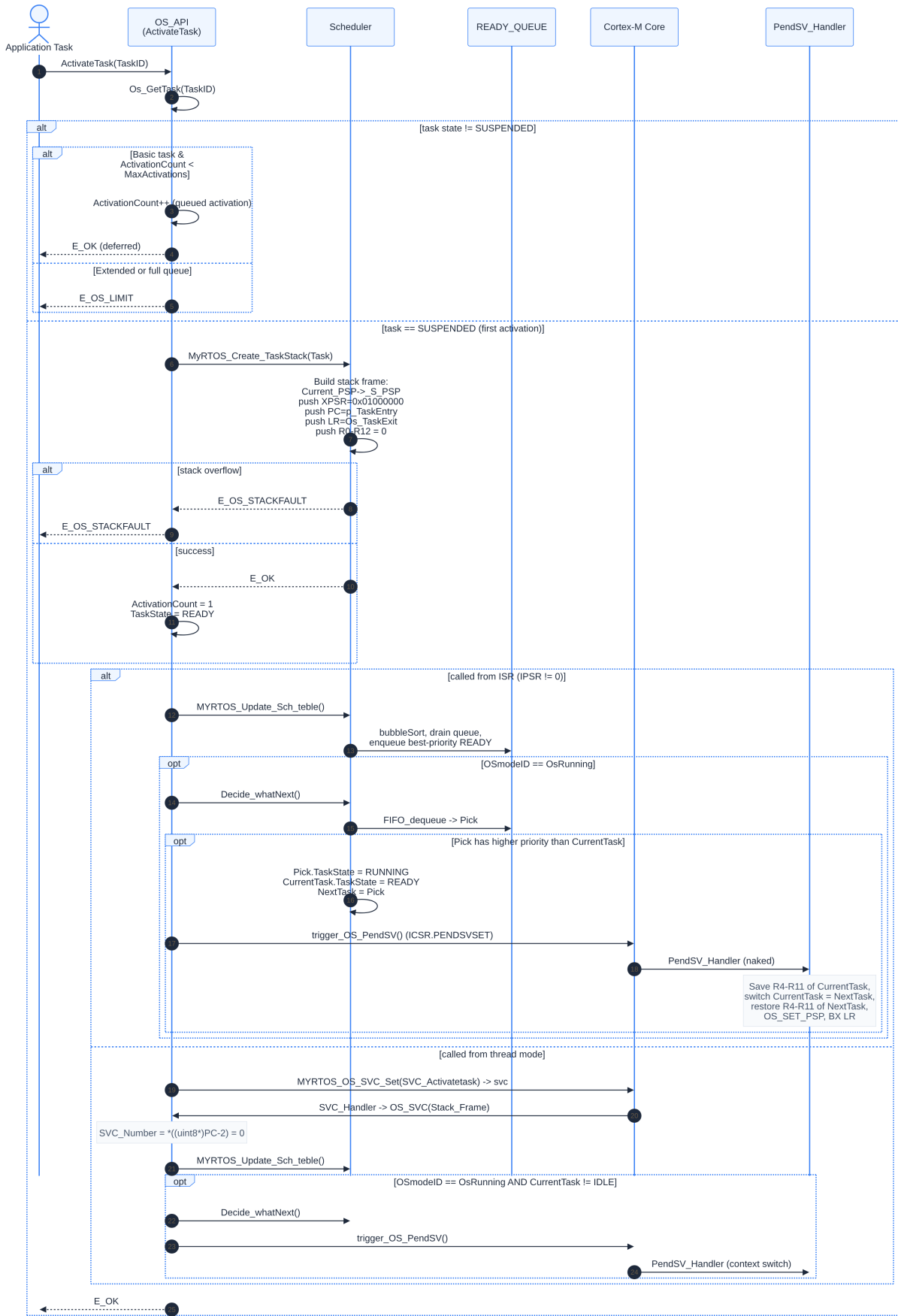
# Sequence Diagram — Start\_OS (Boot)

Boot code calls Start\_OS: main-stack setup, ready-queue init, task-pool creation from TaskConfigTable, AutoStart activations, OSmodeID=OsRunning, Decide\_whatNext, PSP/CONTROL switch to unprivileged thread mode, and finally the first task entry.



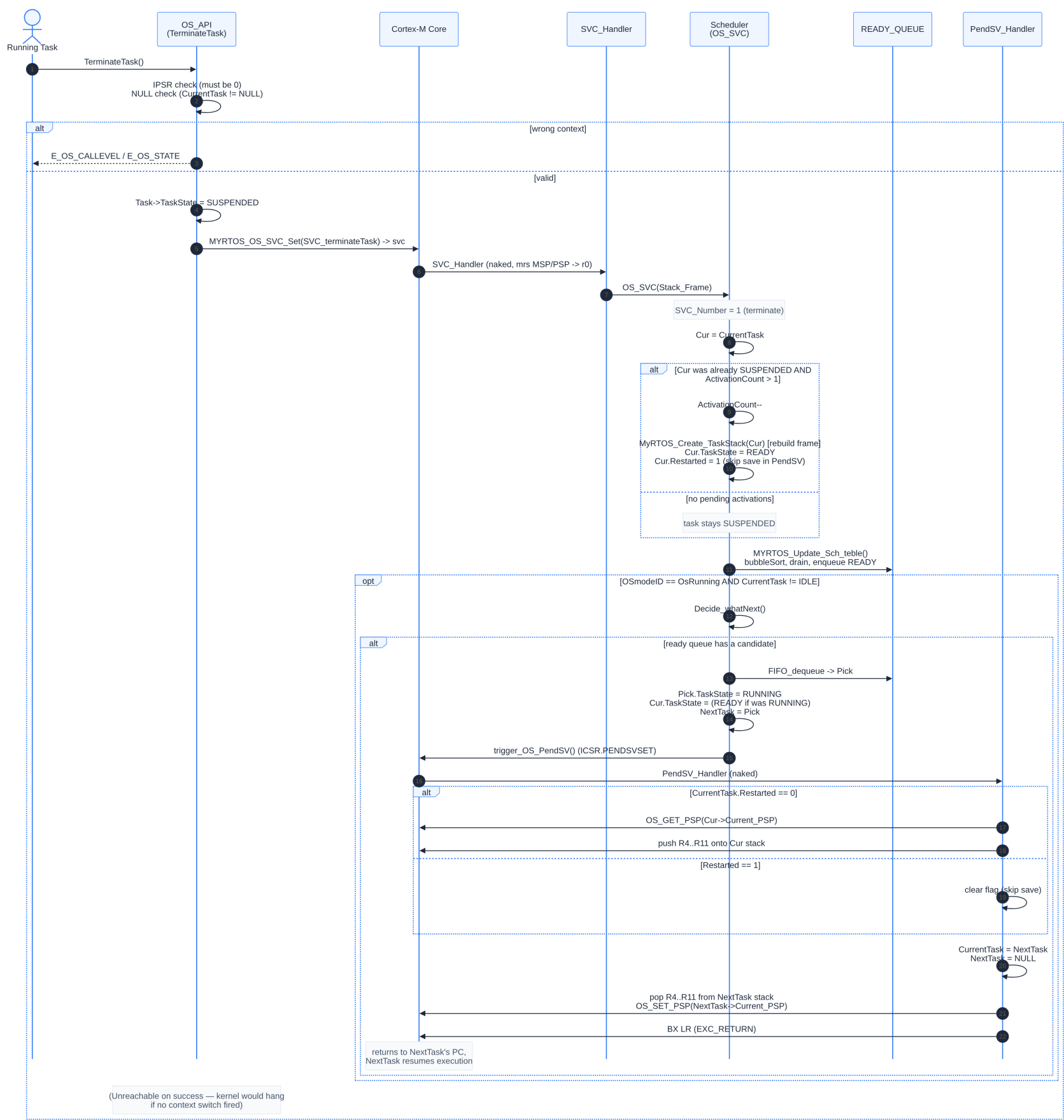
# Sequence Diagram — ActivateTask (thread mode)

Application calls ActivateTask: state checks, basic-task queued-activation counter, stack-frame build (XPSR / PC / LR / R0-R12), ISR-vs-thread branch, SVC #0x00 -> OS\_SVC -> MYRTOS\_Update\_Sch\_tble -> Decide\_whatNext -> PendSV.



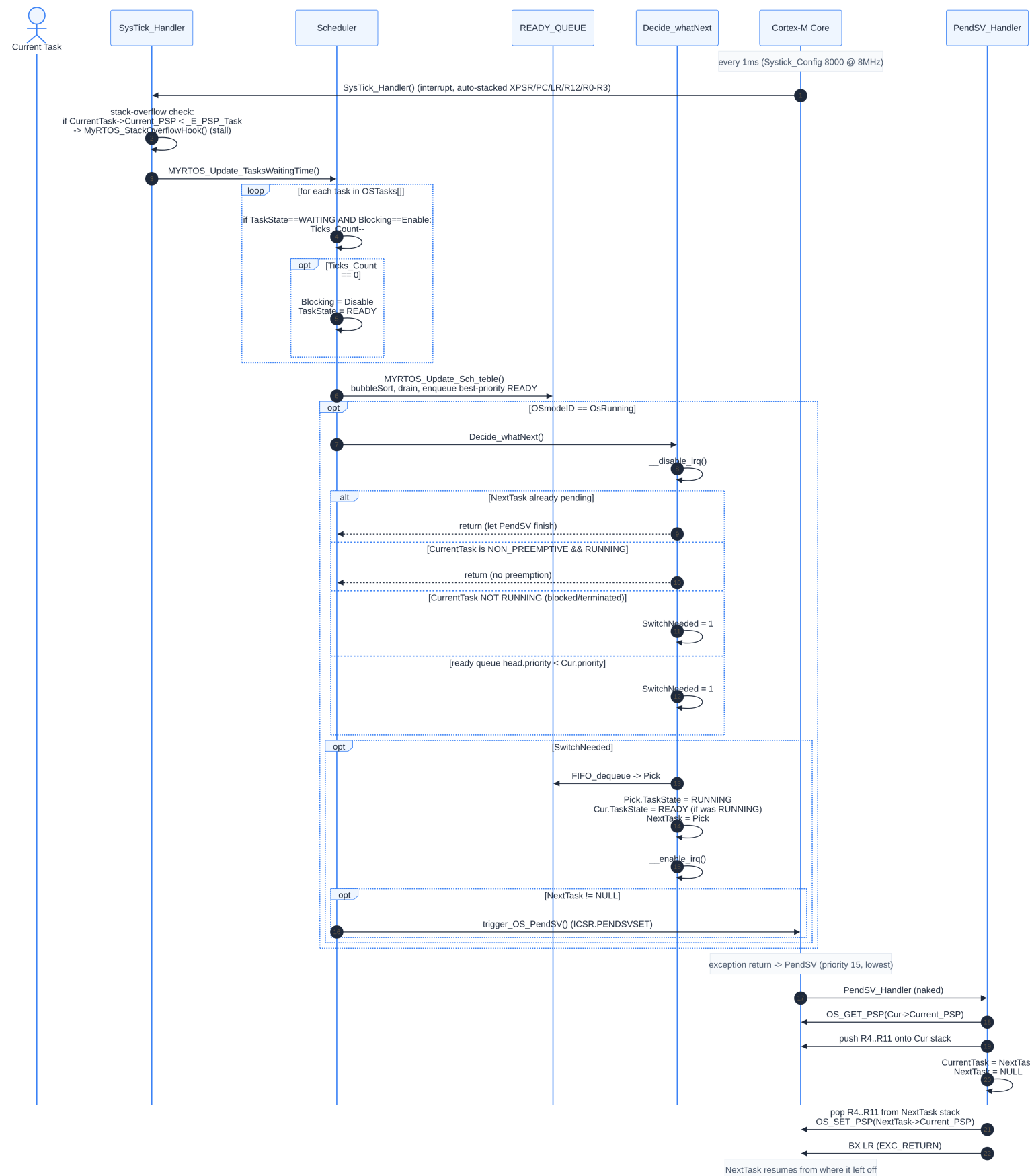
# Sequence Diagram — TerminateTask + PendSV

Current task calls TerminateTask: state = SUSPENDED, SVC #0x01 -> OS\_SVC, Restarted flag handling for basic-task re-activation, ready-queue rebuild, PendSV save R4-R11 / switch CurrentTask=NextTask / restore R4-R11 / BX LR.



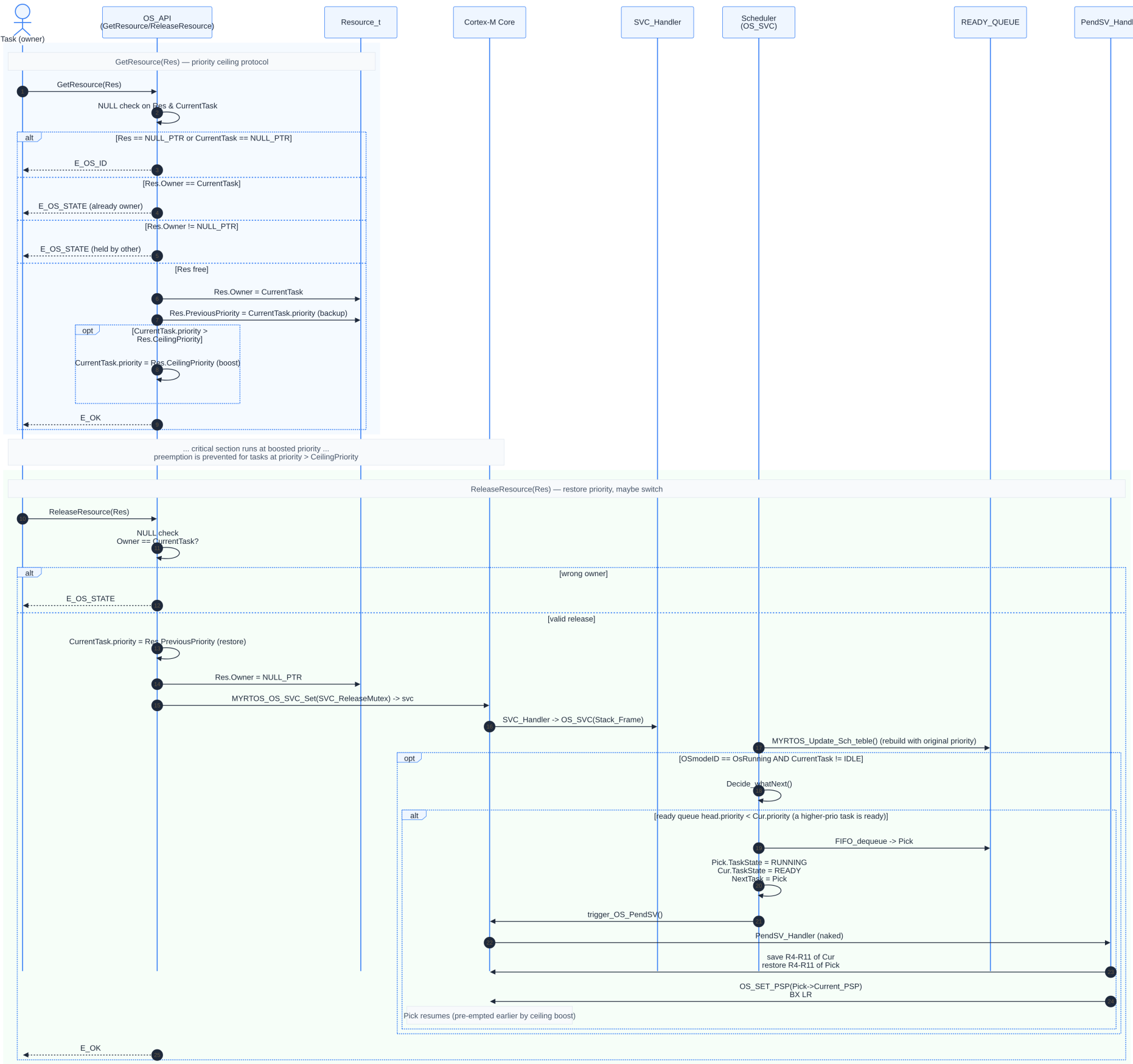
# Sequence Diagram — SysTick Tick (Preemption + Wait Expiry)

Every 1 ms SysTick\_Handler: stack-overflow check, MYRTOS\_Update\_TasksWaitingTime decrements blocked-task counters, ready-queue rebuild, Decide\_whatNext decision tree (non-preemptive guard, blocked/terminated guard, priority-comparison guard), PendSV trigger if a higher-priority task is ready.



# Sequence Diagram — GetResource / ReleaseResource (Priority Ceiling)

GetResource: ownership checks, store PreviousPriority, boost to CeilingPriority. Critical section runs at boosted priority (prevents preemption). ReleaseResource: restore priority, drop ownership, SVC #0x04 -> rebuild queue, Decide\_whatNext may now preempt if a higher-priority task was waiting.



# Sequence Diagram — TaskWait / Os\_Delay

Two branches: (1) block MYSELF -> set WAITING + Ticks\_Count, SVC #0x01 forces immediate context switch; (2) block ANOTHER task -> just dequeue it via SVC #0x02, no switch. SysTick later decrements Ticks\_Count; when it hits 0, the task returns to READY and may preempt.

